



TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCM-UTE

HCMC University of Technology and Engineering

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 2: PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU (02)

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1 (CLO1).** Giải thích được vai trò của mô hình hóa yêu cầu trong CNPM và lý do UML là chuẩn de-facto hiện nay.
2. **MT2 (CLO1, CLO3).** Vẽ được Use Case Diagram đầy đủ cho dự án nhóm: tối thiểu 5 actor, 12 use case, sử dụng đúng 3 loại quan hệ (include, extend, generalization).
3. **MT3 (CLO3).** Sử dụng được công cụ draw.io để vẽ Use Case Diagram đạt chuẩn UML 2.5.
4. **MT4 (CLO1, CLO2).** Phát hiện và sửa được các lỗi điển hình trong Use Case Diagram của nhóm khác (kỹ năng peer review).

NỘI DUNG

1. Mô hình hoá yêu cầu
2. Các phương pháp mô hình hoá truyền thống
3. Ngôn ngữ mô hình hoá thống nhất (UML)
4. Use Case Diagram
5. Quy ước đặt tên và bố cục cho Use Case
6. 10 lỗi điển hình khi vẽ Use Case Diagram
7. Công cụ vẽ

1. Mô hình hoá yêu cầu

- ❑ **Mô hình hóa yêu cầu** (requirements modeling) là quá trình biểu diễn yêu cầu phần mềm dưới dạng đồ họa (sơ đồ, biểu đồ) thay vì chỉ dùng văn bản.
- ❑ **Tại sao cần mô hình hoá?**
- ❑ Theo Sommerville (2016), văn bản tự nhiên có 3 nhược điểm cố hữu khi mô tả yêu cầu phần mềm:
 - ✓ **Mơ hồ**: cùng một câu, mỗi người đọc hiểu một cách khác nhau.
 - ✓ **Khó tổng hợp**: danh sách 50 yêu cầu dạng văn bản khó nhìn ra bức tranh tổng thể.
 - ✓ **Khó phát hiện thiếu sót**: không nhìn thấy ngay yêu cầu nào còn thiếu.

1. Mô hình hoá yêu cầu

- ❑ Mô hình đồ họa khắc phục cả 3 nhược điểm trên:
 - ✓ Ký hiệu chuẩn (UML) → ai cũng hiểu giống nhau.
 - ✓ Một sơ đồ thể hiện toàn cảnh trên một trang.
 - ✓ Khi vẽ, dễ phát hiện “actor này còn cần làm gì nữa không?”
- ❑ Lưu ý: Mô hình hóa **KHÔNG** thay thế văn bản. Nó **BỔ SUNG**. Một dự án thật có cả tài liệu văn bản (SRS, User Story) **VÀ** sơ đồ (UML).

1. Mô hình hoá yêu cầu

Phân biệt 4 loại model trong CNPM

Loại model	Mục đích	Ví dụ lược đồ UML
Context model	Hiển thị ranh giới hệ thống và môi trường xung quanh	Use Case Diagram (cấp tổng quan)
Interaction model	Mô tả tương tác giữa user-system hoặc giữa các thành phần	Use Case Diagram, Sequence Diagram
Structural model	Mô tả cấu trúc tĩnh của hệ thống (lớp, quan hệ)	Class Diagram, Component Diagram
Behavioral model	Mô tả hành vi động của hệ thống	State Diagram, Activity Diagram

2. Các phương pháp mô hình hóa truyền thống

Trước khi UML ra đời (1997), các phương pháp mô hình hóa khác đã được sử dụng phổ biến. Hai phương pháp tiêu biểu được sử dụng:

1. DFD - Data Flow Diagram (Sơ đồ luồng dữ liệu)
2. ERD - Entity Relationship Diagram

2. Các phương pháp mô hình hóa truyền thống

❑ DFD - Data Flow Diagram (Sơ đồ luồng dữ liệu)

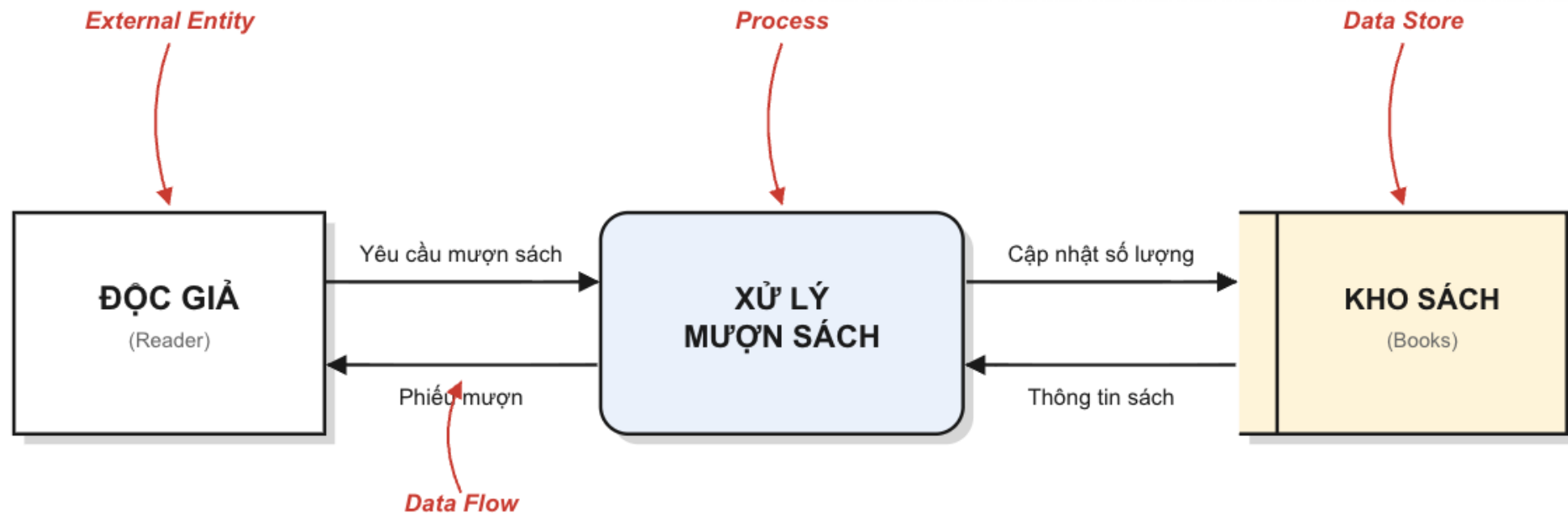
Phát triển bởi Larry Constantine và Ed Yourdon trong những năm 1970s.

Mô tả luồng DỮ LIỆU di chuyển trong hệ thống qua các tiến trình xử lý.

❑ 4 ký hiệu cơ bản:

- ✓ Process (hình tròn hoặc bo góc): tiến trình xử lý dữ liệu.
- ✓ Data store (hình chữ nhật hở 2 đầu): nơi lưu trữ dữ liệu.
- ✓ External entity (hình chữ nhật): tác nhân ngoài hệ thống.
- ✓ Data flow (mũi tên): luồng dữ liệu di chuyển.

2. Các phương pháp mô hình hóa truyền thống



Lưu ý:

Mọi luồng dữ liệu đều phải đi qua một process.

Không nối trực tiếp Entity-Entity hay Entity-Data Store.

2. Các phương pháp mô hình hóa truyền thống

ERD - Entity Relationship Diagram

Phát triển bởi Peter Chen năm 1976. Mô tả cấu trúc DỮ LIỆU - các thực thể (entity) và mối quan hệ (relationship) giữa chúng. ERD là tiền thân của Class Diagram trong UML.

VỊ TRÍ HIỆN TẠI CỦA DFD VÀ ERD

- DFD hầu như KHÔNG còn được dùng trong industry hiện đại, nó đã được thay thế bởi Sequence Diagram, Activity Diagram, và Component Diagram của UML.
- ERD vẫn được dùng phổ biến, đặc biệt khi thiết kế cơ sở dữ liệu quan hệ. UML Class Diagram có thể thay thế ERD nhưng nhiều DBA vẫn quen dùng ERD truyền thống.

3. Ngôn ngữ mô hình hoá thống nhất (UML)

UML là gì?

UML (Unified Modeling Language - Ngôn ngữ Mô hình hóa Thống nhất) là một chuẩn quốc tế cho việc đặc tả, hình dung, xây dựng và tài liệu hóa các sản phẩm phần mềm.

3. Ngôn ngữ mô hình hoá thống nhất (UML)

Lịch sử phát triển:

- ✓ **1990s đầu:** Có 3 phương pháp mô hình hóa Object-Oriented chính cạnh tranh: OMT (Rumbaugh), Booch (Booch), OOSE (Jacobson).
- ✓ **1995:** “Three Amigos” (Rumbaugh, Booch, Jacobson) hợp tác tại Rational Software, hợp nhất 3 phương pháp thành UML.
- ✓ **1997:** UML 1.0 được Object Management Group (OMG) phê chuẩn làm chuẩn.
- ✓ **2005:** UML 2.0 - bản đại tu, có 13 loại lược đồ.
- ✓ **2017:** UML 2.5.1 (phiên bản hiện hành) - 14 loại lược đồ.

3. Ngôn ngữ mô hình hoá thống nhất (UML)

14 loại lược đồ UML

Theo OMG UML 2.5.1, có 14 loại lược đồ chia làm 2 nhóm chính:

Nhóm	Lược đồ	Mục đích
Structural (Cấu trúc tĩnh)	Class Diagram	Lớp và quan hệ giữa các lớp
	Object Diagram	Instance cụ thể của các lớp tại một thời điểm
	Component Diagram	Thành phần phần mềm và quan hệ
	Composite Structure Diagram	Cấu trúc nội bộ của một classifier
	Deployment Diagram	Hạ tầng vật lý nơi phần mềm chạy
	Package Diagram	Tổ chức các package
	Profile Diagram	Mở rộng UML cho domain cụ thể

3. Ngôn ngữ mô hình hoá thống nhất (UML)

14 loại lược đồ UML

Theo OMG UML 2.5.1, có 14 loại lược đồ chia làm 2 nhóm chính:

Nhóm	Lược đồ	Mục đích
Behavioral (Hành vi động)	Use Case Diagram	Chức năng hệ thống từ góc nhìn user
	Activity Diagram	Luồng công việc, business process
	State Machine Diagram	Trạng thái và chuyển trạng thái của object
	Sequence Diagram	Tương tác giữa object theo thời gian
	Communication Diagram	Tương tác giữa object theo cấu trúc
	Timing Diagram	Tương tác theo thời gian thực
	Interaction Overview Diagram	Tổng quan các interaction

4. Use Case Diagram

Use Case Diagram được Ivar Jacobson giới thiệu năm 1986 trong phương pháp OOSE, sau trở thành một phần của UML. Đây là loại lược đồ ĐẦU TIÊN cần vẽ trong dự án - nó định hình toàn bộ phạm vi hệ thống.

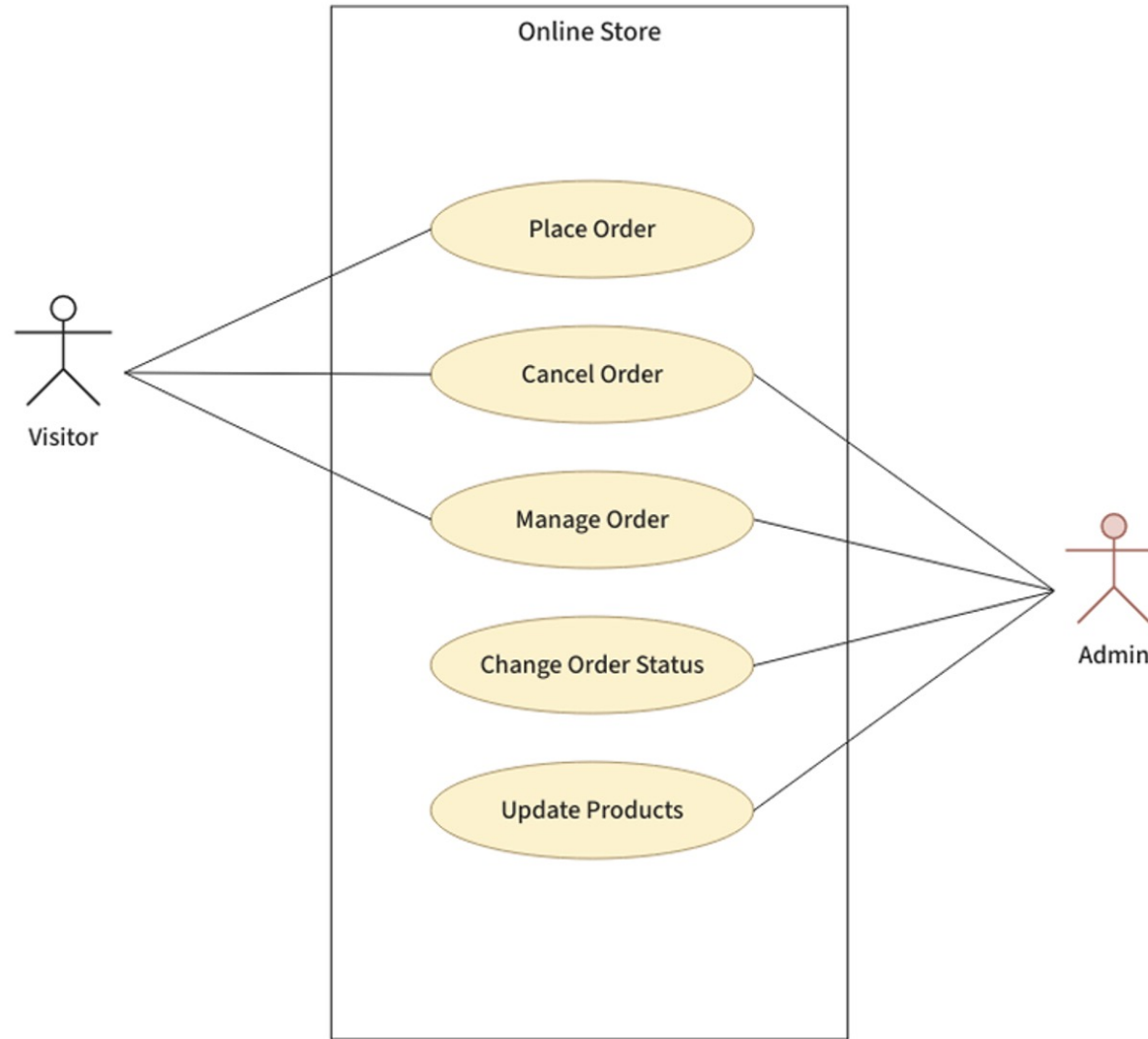
❑ **Định nghĩa** Theo OMG UML 2.5.1:

“A use case diagram is a graph of actors, a set of use cases enclosed by a system boundary, communication associations between the actors and the use cases, and generalization among the use cases.”

❑ **Diễn giải đơn giản:** Use Case Diagram trả lời 2 câu hỏi:

- ✓ Ai (actor) sẽ tương tác với hệ thống?
- ✓ Họ làm gì (use case) với hệ thống?

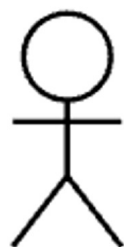
4. Use Case Diagram



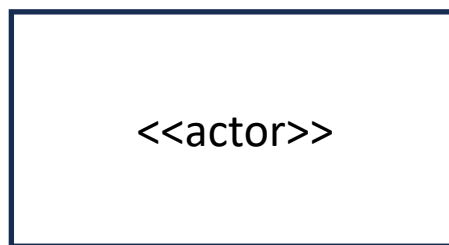
4. Use Case Diagram

5 thành phần cơ bản

- ❑ **Thành phần 1: Actor (Tác nhân)**
- ❑ **Định nghĩa:** Một vai trò mà một người, một hệ thống ngoài, hoặc một thiết bị đóng khi tương tác với hệ thống.
- ❑ **Ký hiệu:** Stick figure (hình người que) - quy ước phổ biến nhất. Có thể dùng hình chữ nhật với <<actor>> stereotype cho hệ thống ngoài.



Actor



4. Use Case Diagram

❑ Thành phần 1: Actor (Tác nhân)

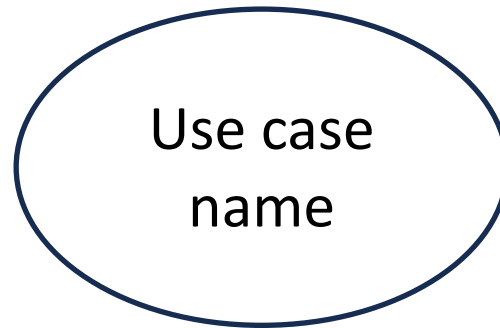
❑ Quy ước:

- ✓ Actor là VAI TRÒ, không phải con người cụ thể. Một người có thể đóng nhiều vai trò (Anh Nam vừa là Chủ quán vừa là Thu ngân khi cần).
- ✓ Tên actor là DANH TỪ, viết hoa chữ đầu. Vd: “Thu ngân”, “Chủ quán”, không phải “thu_ngan”.
- ✓ Đặt actor BÊN NGOÀI ranh giới hệ thống. Primary actor bên trái, Secondary bên phải (quy ước, không bắt buộc).

4. Use Case Diagram

❑ Thành phần 2: Use Case (Trường hợp sử dụng)

- ✓ Định nghĩa: Một chuỗi hành động mà hệ thống thực hiện để đem lại một kết quả có giá trị cho actor.
- ✓ Ký hiệu: Hình elip (oval), bên trong là tên use case.



4. Use Case Diagram

❑ Thành phần 2: Use Case (Trường hợp sử dụng)

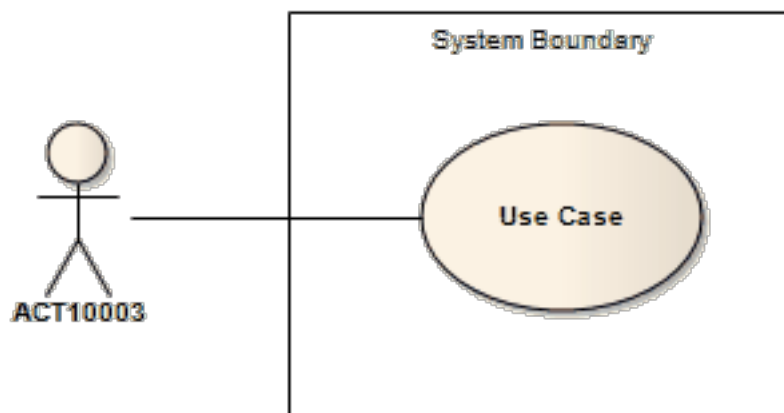
❑ Quy ước:

- ✓ Bắt đầu bằng động từ + danh từ. Vd: “Tạo đơn”, “In hóa đơn”, “Xem báo cáo”.
- ✓ Mô tả NHIỆM VỤ NGHIỆP VỤ, không phải thao tác kỹ thuật. “Tạo đơn” đúng. “INSERT vào bảng Order” sai.
- ✓ Mức độ chi tiết phù hợp: KHÔNG quá to (“Quản lý hệ thống”) và KHÔNG quá nhỏ (“Click nút Save”).

4. Use Case Diagram

❑ Thành phần 3: System Boundary (Ranh giới hệ thống)

- ✓ **Ký hiệu:** Hình chữ nhật bao quanh tất cả use case của hệ thống. Tên hệ thống ghi ở góc trên hoặc giữa cạnh trên.
- ✓ **Ý nghĩa:** Phân biệt CÁI GÌ thuộc hệ thống ta đang xây (use case nằm trong hộp) và CÁI GÌ ngoài hệ thống (actor nằm ngoài hộp).



4. Use Case Diagram

❑ Thành phần 3: System Boundary (Ranh giới hệ thống)

❑ Quy ước:

- ✓ Phải có ranh giới hệ thống. Không có là một lỗi nghiêm trọng.
- ✓ Use case LUÔN nằm bên trong, actor LUÔN nằm bên ngoài.
- ✓ Tên ranh giới = tên hệ thống đang xây. Vd: “Hệ thống Quản lý Quán cà phê”.

4. Use Case Diagram

❑ Thành phần 4: Association (Quan hệ tương tác)

- ✓ **Ký hiệu:** Đường thẳng (không có mũi tên hoặc có mũi tên một chiều) nối actor với use case.
- ✓ **Ý nghĩa:** Actor tham gia vào use case này. KHÔNG có ý nghĩa về “luồng” hay “thứ tự”, chỉ là tham gia.



4. Use Case Diagram

❑ Thành phần 4: Association (Quan hệ tương tác)

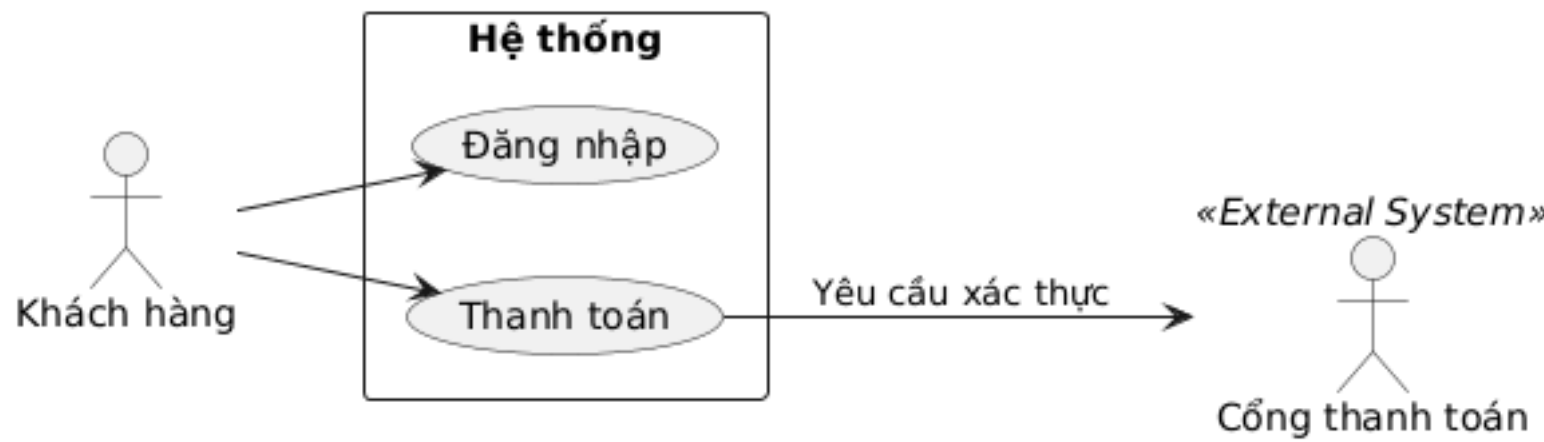
❑ Quy ước:

- ✓ Đường thẳng đơn giản, KHÔNG đặt tên trên đường (trừ khi có nhiều association giữa 1 cặp actor-use case).
- ✓ Có thể dùng mũi tên một chiều để chỉ ai khởi động (primary): mũi tên TỪ actor ĐẾN use case.
- ✓ KHÔNG có association giữa 2 actor (trừ generalization), KHÔNG có association giữa 2 use case có dạng đường thẳng (chỉ có include/extend/generalization có dạng mũi tên đứt nét).

4. Use Case Diagram

❑ Thành phần 4: Association (Quan hệ tương tác)

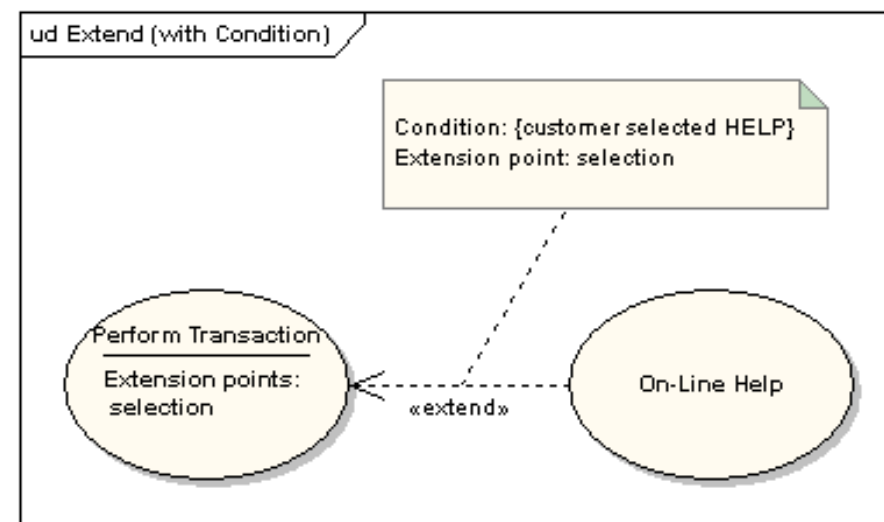
- ✓ **Lưu ý:** Mũi tên được dùng để chỉ rõ hướng bắt đầu của tương tác
- ✓ Mũi tên từ Actor đến Use Case: Actor là người khởi xướng (Primary Actor) hành động.
- ✓ Mũi tên từ UC đến Actor: Dùng cho các Tác nhân phụ (Secondary Actor/Supporting Actor), thường là các hệ thống bên ngoài.



4. Use Case Diagram

❑ Thành phần 5: Note (Ghi chú) - tùy chọn

- ✓ **Ký hiệu:** Hình chữ nhật bị gấp một góc, nối với một thành phần bằng đường đứt nét.
- ✓ **Mục đích:** Thêm chú thích để làm rõ một thành phần (vd: business rule, ràng buộc, giả định).
- ✓ **Quy ước:** Dùng ít. Quá nhiều note làm sơ đồ rối. Nếu cần giải thích nhiều, viết Use Case Specification.



4. Use Case Diagram

❑ 3 quan hệ chính của Use Case

Ngoài association (actor ↔ use case), UML định nghĩa 3 loại quan hệ giữa các use case với nhau, hoặc giữa các actor với nhau:

- ✓ Quan hệ 1: <<include>> (Bao gồm)
- ✓ Quan hệ 2: <<extend>> (Mở rộng)
- ✓ Quan hệ 3: Generalization (Tổng quát hóa / Kế thừa)

4. Use Case Diagram

❑ Quan hệ 1: <<include>> (Bao gồm)

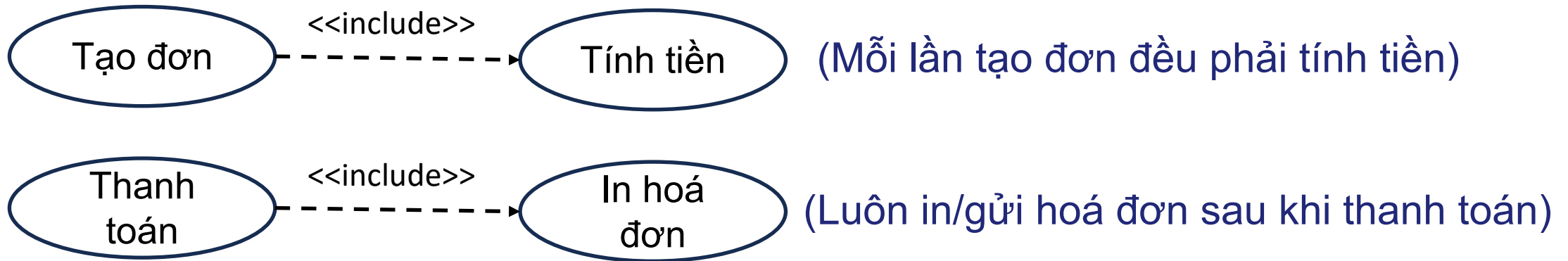
- ✓ **Định nghĩa** theo OMG UML 2.5.1: “An include relationship is a relationship between two use cases, indicating that the behavior of the included use case is inserted into the behavior of the including use case.”
- ✓ **Diễn giải:** Use case A include Use case B nghĩa là: A LUÔN cần B để hoàn thành. B là phần **BẮT BUỘC** trong luồng của A.
- ✓ **Ký hiệu:**



4. Use Case Diagram

❑ Quan hệ 1: <<include>> (Bao gồm)

✓ Ví dụ Quán cà phê:



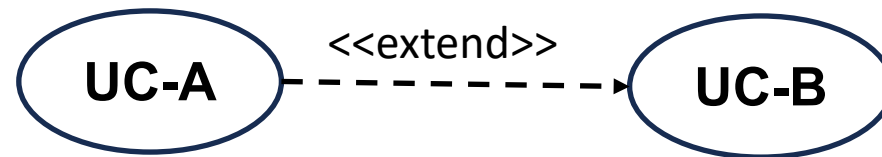
✓ Khi nào dùng <<include>>:

- Khi một hành vi LẶP LẠI ở nhiều use case → tách ra để tránh trùng lặp.
- Khi muốn chia một use case lớn thành các phần nhỏ rõ ràng.

4. Use Case Diagram

❑ Quan hệ 2: <<extend>> (Mở rộng)

- ✓ **Định nghĩa** theo OMG UML 2.5.1: “An extend relationship is a relationship from an extending use case to an extended use case that specifies how and when the behavior defined in the extending use case can be inserted into the behavior defined in the extended use case.
- ✓ **Diễn giải:** Use case A extend Use case B nghĩa là: A là hành vi BỔ SUNG TÙY CHỌN cho B, chỉ chạy khi đáp ứng điều kiện nhất định.
- ✓ **Ký hiệu:**

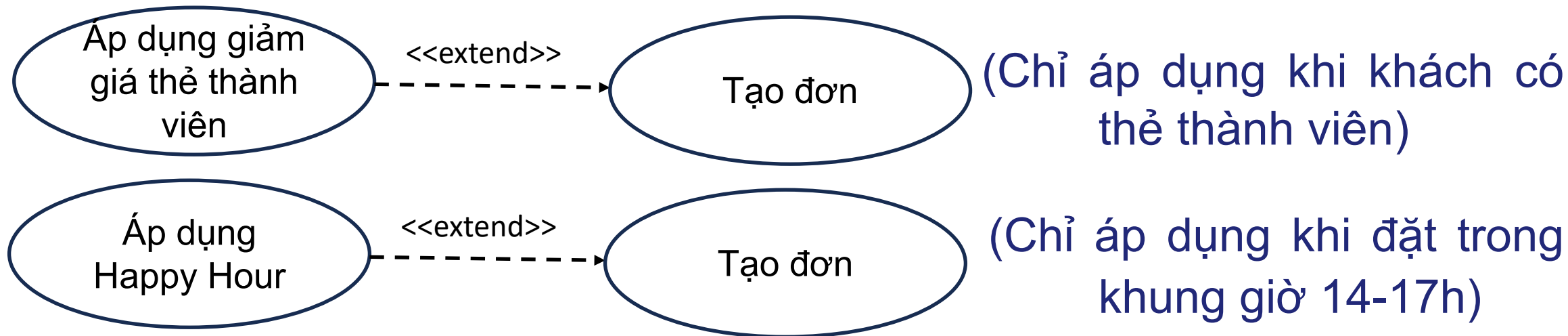


(mũi tên **ĐÚT NÉT**, từ A đến B → A mở rộng B)

4. Use Case Diagram

❑ Quan hệ 2: <<extend>> (Mở rộng)

✓ Ví dụ Quán cà phê:



✓ Khi nào dùng <<extend>>:

- Khi một hành vi CHỈ THỈNH THOẢNG xảy ra (có điều kiện).
- Khi muốn mô hình hóa các tính năng tùy chọn (optional features).

4. Use Case Diagram

❑ Phân biệt <<include>> vs <<extend>>

- ✓ <<include>> = LUÔN LUÔN xảy ra (mandatory)
- ✓ <<extend>> = THỈNH THOẢNG xảy ra (optional)
- ✓ <<include>>: Mũi tên TỪ use case GỌI đến use case BỊ GỌI. (“Tạo đơn” gọi “Tính tiền” → mũi tên từ Tạo đơn đến Tính tiền)
- ✓ <<extend>>: Mũi tên TỪ use case MỞ RỘNG đến use case CƠ BẢN. (“Áp dụng giảm giá” mở rộng “Tạo đơn” → mũi tên từ “Áp dụng giảm giá” đến “Tạo đơn”)

→ Hai mũi tên ngược chiều nhau!

4. Use Case Diagram

❑ Quan hệ 3: Generalization (Tổng quát hóa / Kế thừa)

- ✓ **Định nghĩa:** Quan hệ kế thừa – “Sub” kế thừa và có thể bổ sung từ “Super”. Áp dụng được cho cả Use Case và Actor
- ✓ **Ký hiệu:**

Sub —————> Super

(mũi tên ĐẦU TAM GIÁC RỘNG, từ Sub đến Super)

- ✓ **Ví dụ Generalization giữa Actor (Quán cà phê):**

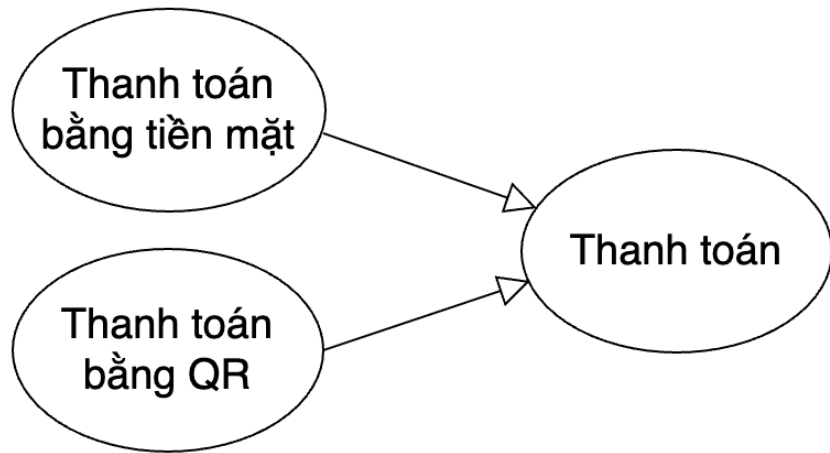


Khách hàng VIP có TẤT CẢ tính năng của Khách hàng + thêm các tính năng riêng (giảm giá tự động, tích điểm).

4. Use Case Diagram

❑ Quan hệ 3: Generalization (Tổng quát hóa / Kế thừa)

✓ Ví dụ Generalization giữa Use Case (Quán cà phê):



“Thanh toán bằng tiền mặt” và “Thanh toán bằng QR” đều là cách cụ thể của thanh toán.

✓ Khi nào dùng Generalization?

- Khi có nhiều actor/use case có chung tính năng và một số tính năng riêng.
- Khi muốn mô hình hóa hierarchy (cha - con) trong domain.

4. Use Case Diagram

Tóm tắt 3 mối quan hệ

Quan hệ	Ký hiệu	Ý nghĩa	Ví dụ
<<include>>	Mũi tên đứt nét ----->	Bao gồm bắt buộc	Tạo đơn --> Tính tiền
<<extend>>	Mũi tên đứt nét ----->	Mở rộng có điều kiện	Áp dụng giảm giá --> Tạo đơn
Generalization	Mũi tên đầu tam giác rỗng ————>	Kế thừa	Khách hàng VIP —> Khách hàng

5. Quy ước đặt tên và bố cục cho Use Case

- ❑ **Quy ước đặt tên Use Case:** Một use case được đặt tên đúng phải đáp ứng cả 4 tiêu chí của Larman (2004) và Cockburn (2000):
 1. **Bắt đầu bằng động từ chỉ hành động.** Đúng: “Tạo đơn”, “Xem báo cáo”. Sai: “Đơn hàng”, “Báo cáo” (danh từ).
 2. **Mô tả MỤC TIÊU NGHIỆP VỤ.** Đúng: “Thanh toán đơn”. Sai: “Click nút Pay” (thao tác UI), “INSERT vào DB” (thao tác kỹ thuật).
 3. **Cụ thể, không mơ hồ.** Sai: “Quản lý món” (quá rộng - quản lý là gì? thêm? sửa? xóa?). Đúng: tách thành “Thêm món mới”, “Sửa thông tin món”, “Xóa món”.
 4. **Có thể đọc to và khách hàng hiểu.** Test: đọc tên use case cho anh Nam (chủ quán) - anh có hiểu ngay không? Nếu phải giải thích thì tên chưa tốt.

5. Quy ước đặt tên và bố cục cho Use Case

❑ **Bố cục Use Case Diagram**

Quy ước phổ biến (không bắt buộc nhưng giúp sơ đồ dễ đọc):

- ✓ Primary actors: bên TRÁI ranh giới hệ thống.
- ✓ Secondary actors: bên PHẢI ranh giới hệ thống.
- ✓ Use case quan trọng nhất: ở GIỮA hộp.
- ✓ Use case liên quan: nhóm gần nhau (vd: tất cả use case về “thanh toán” cùng một vùng).
- ✓ Đường nối tránh cắt nhau hết mức có thể.

5. Quy ước đặt tên và bố cục cho Use Case

❑ Quy mô Use Case Diagram

Một sơ đồ tốt có:

- ✓ Tối thiểu 3 actor (1 primary + 2 khác).
- ✓ Tối thiểu 8 - 10 use case.
- ✓ Tối đa 20 - 25 use case trên 1 sơ đồ - nhiều hơn nên CHIA THÀNH NHIỀU SƠ ĐỒ theo subsystem (vd: “UC bán hàng”, “UC quản lý kho”, “UC báo cáo”).
- ✓ Tỷ lệ phù hợp: 70 - 80% số quan hệ là association (actor ↔ use case), 20-30% là include/extend/generalization.

6. 10 lỗi điển hình khi vẽ Use Case Diagram

❑ Lỗi 1: Đặt tên use case dạng danh từ

- ✓ Sai: “Đơn hàng”, “Báo cáo doanh thu”.
- ✓ Đúng: “Tạo đơn hàng”, “Xem báo cáo doanh thu”.

❑ Lỗi 2: Bỏ system boundary

- ✓ Sai: Vẽ actor và use case nhưng không có khung bao quanh use case.
- ✓ Hậu quả: Không phân biệt được cái gì thuộc hệ thống, cái gì ngoài.

6. 10 lỗi điển hình khi vẽ Use Case Diagram

❑ Lỗi 3: Vẽ flow giữa các use case bằng mũi tên thường

- ✓ Sai: “Tạo đơn” → mũi tên thường → “Tính tiền” → mũi tên thường → “Thanh toán” (như sơ đồ luồng).
- ✓ Lỗi này phổ biến nhất. Use Case Diagram KHÔNG mô tả thứ tự hay luồng. Đó là nhiệm vụ của Activity Diagram hoặc Sequence Diagram.
- ✓ Sửa: Nếu “Tạo đơn” cần “Tính tiền” → dùng <<include>>. Còn “Thanh toán” là use case riêng do actor khởi động sau.

6. 10 lỗi điển hình khi vẽ Use Case Diagram

❑ Lỗi 4: Lẫn lộn <<include>> và <<extend>>

- ✓ Sai: “Tạo đơn” <<extend>> “Tính tiền” (Tính tiền không phải optional).
- ✓ Đúng: “Tạo đơn” <<include>> “Tính tiền” (luôn cần). Còn “Áp dụng giảm giá” <<extend>> “Tạo đơn” (chỉ khi có điều kiện).

❑ Lỗi 5: Sai chiều mũi tên <<include>> hoặc <<extend>>

- ✓ Sai: “Tính tiền” <<include>> “Tạo đơn” (mũi tên ngược).
- ✓ Đúng: “Tạo đơn” <<include>> “Tính tiền” (use case GỌI chỉ vào use case BỊ GỌI).

6. 10 lỗi điển hình khi vẽ Use Case Diagram

❑ Lỗi 6: Có quan hệ giữa 2 actor (sai, ngoài generalization)

- ✓ Sai: Vẽ đường thẳng giữa “Thu ngân” và “Chủ quán” để thể hiện “Thu ngân báo cáo cho Chủ quán”.
- ✓ Đúng: Quan hệ giữa actor chỉ có Generalization. Việc “Thu ngân báo cáo cho Chủ quán” là use case (“Báo cáo”), không phải quan hệ trực tiếp giữa actor.

❑ Lỗi 7: Đưa thao tác kỹ thuật làm use case

- ✓ Sai: “Mở kết nối DB”, “Validate input”, “Mã hóa mật khẩu”.
- ✓ Đúng: Đây là chi tiết kỹ thuật của implementation, không phải mục tiêu của user. User không quan tâm DB connection, họ quan tâm “Đăng nhập” hay “Tạo đơn”.

6. 10 lỗi điển hình khi vẽ Use Case Diagram

❑ Lỗi 8: Quá ít hoặc quá nhiều use case

- ✓ Quá ít (< 5 use case): Hệ thống chưa được khám phá đủ. Thường vì sinh viên gộp “Quản lý X” thành 1 use case.
- ✓ Quá nhiều ($> 25-30$ use case): Quá chi tiết, đến mức hệ thống đó nên chia thành các subsystem.

❑ Lỗi 9: Bỏ sót Secondary Actor

- ✓ Sai: Chỉ có người dùng (thu ngân, chủ quán) làm actor.
- ✓ Đúng: Bao gồm cả hệ thống ngoài tương tác. Ví dụ: “Cổng VNPay”, “Hệ thống email”, “Máy in”.

6. 10 lỗi điển hình khi vẽ Use Case Diagram

❑ Lỗi 10: Use case mà khách hàng không hiểu

- ✓ Test cuối: Đưa cho anh Nam (chủ quán) đọc Use Case Diagram. Nếu anh không hiểu được 80% nội dung → sơ đồ chưa đạt.
- ✓ Use Case Diagram là CẦU NỐI giao tiếp với khách hàng. Nếu khách hàng không đọc được, sơ đồ thất bại mục đích chính.

7. Công cụ vẽ

Có nhiều công cụ vẽ Use Case Diagram. Bảng so sánh 3 công cụ phổ biến nhất:

Tiêu chí	draw.io	Enterprise Architect	StarUML
Chi phí	Miễn phí	Thương mại (~250 USD/license)	Miễn phí có giới hạn
Cài đặt	Web-based, không cần cài	Cần cài Windows/Mac	Cần cài Windows/Mac/Linux
Độ dễ học	Rất dễ, kéo thả	Trung bình, UI phức tạp	Dễ, UI gần với draw.io
Thư viện UML	Đầy đủ 14 lược đồ UML	Đầy đủ + nhiều mở rộng	Đầy đủ 14 lược đồ
Sinh code	Không	Có (Java, C#, C++, ...)	Có (giới hạn)
Cộng tác	Tốt (Google Drive, GitHub)	Tốt (server riêng)	Hạn chế
Phù hợp với	Lớp học, dự án nhỏ-vừa	Doanh nghiệp lớn, dự án nhiều người	Trung gian

THỰC HÀNH

1. **Chuyển yêu cầu thành Use Case:** Mỗi nhóm phân tích danh sách yêu cầu của Buổi 2, xác định Actor và Use Case ứng cử viên.
2. **Sử dụng draw.io vẽ tất cả Use Case Diagram**

HƯỚNG DẪN THỰC HÀNH

1. Chuyển yêu cầu thành Use Case

❑ Mục tiêu:

- ✓ Xác định được Actor (primary + secondary) từ danh sách yêu cầu đã có.
- ✓ Xác định được tập Use Case ứng cử viên.
- ✓ Phân biệt được “yêu cầu” và “use case” → không phải mỗi yêu cầu là một use case.

HƯỚNG DẪN THỰC HÀNH

1. Chuyển yêu cầu thành Use Case

□ Giải thích:

- ✓ Một yêu cầu KHÔNG nhất thiết là một use case. Nhiều yêu cầu có thể gộp vào 1 use case (vd: 'tính tiền' và 'in hóa đơn' đều thuộc use case 'Thanh toán'). Một yêu cầu cũng có thể là NFR → không thành use case riêng (vd: 'phản hồi dưới 1 giây' không phải use case)."
- ✓ Quy tắc: chỉ FUNCTIONAL REQUIREMENT mới có thể trở thành use case. NFR là ràng buộc của các use case, không phải use case riêng.

HƯỚNG DẪN THỰC HÀNH

1. Chuyển yêu cầu thành Use Case

❑ **Bước 1: Xác định Actor.** Liệt kê tất cả “vai trò” sẽ tương tác với hệ thống:

- ✓ Nhìn vào cột “As a [vai trò]” trong các User Story → liệt kê tất cả vai trò xuất hiện.
- ✓ Đọc lại Persona → bổ sung các vai trò chưa có trong User Story.
- ✓ Nghĩ về Secondary Actor: Hệ thống nào ngoài cần tương tác? (cổng thanh toán, máy in, hệ thống email...)
- ✓ Phân loại Primary vs Secondary cho mỗi actor.
- ✓ Ghi vào cột “Actor”.

HƯỚNG DẪN THỰC HÀNH

1. Chuyển yêu cầu thành Use Case

- ❑ **Bước 2: Trích xuất UC.** Với mỗi yêu cầu Functional trong danh sách:
 - ✓ Hỏi: “Đây là một MỤC TIÊU NGHIỆP VỤ độc lập, hay là chi tiết của một mục tiêu lớn hơn?”
 - ✓ Nếu mục tiêu độc lập → là 1 use case. Đặt tên động từ + danh từ.
 - ✓ Nếu là chi tiết → gộp vào use case lớn hơn (ghi nhận như là một bước trong main flow).
 - ✓ Ghi vào cột “Use Case”.

HƯỚNG DẪN THỰC HÀNH

1. Chuyển yêu cầu thành Use Case

- ❑ **Bước 3: Mapping Actor ↔ Use Case.** Với mỗi use case, vẽ đường thẳng nối với actor sẽ KHỞI ĐỘNG nó (Primary actor):
 - ✓ Một use case CÓ THỂ có nhiều actor (vd: “Tạo đơn” có thể do Thu ngân HOẶC Khách hàng (đặt online) khởi động).
 - ✓ Một actor có nhiều use case là bình thường. Thu ngân khởi động Tạo đơn, Hủy đơn, Sửa đơn...

HƯỚNG DẪN THỰC HÀNH

1. Chuyển yêu cầu thành Use Case

❑ Tự kiểm tra

- ✓ Có ít nhất 5 actor không?
- ✓ Có ít nhất 12 use case không?
- ✓ Có actor nào không có use case nào không? (nếu có → cần xem lại)
- ✓ Có use case nào không có actor nào khởi động không? (nếu có → cần xem lại)

BÀI TẬP VỀ NHÀ

Yêu cầu:

- ✓ Mở rộng UC Diagram: tối thiểu 16 use case.
- ✓ Phải có ít nhất 4 quan hệ <<include>>, 4 <<extend>>, và 2 generalization.
- ✓ Chọn 8-10 use case quan trọng nhất từ UC Diagram để chuẩn bị viết Use Case Specification. Đánh dấu 8-10 use case đó bằng màu đặc biệt trên sơ đồ.
- ✓ Nộp file .drawio + ảnh PNG + danh sách 8-10 use case ưu tiên (file Word/text) lên thư mục Drive lớp.